



COMPANY OR INSTITUTION NAME

Department, Team or Reference Sector

Technical Report / Project Report

Project Reference, Client or Exam

EngTeX — Engineering L^AT_EX

Template

User Manual

Author:

Luca Gaetano Sansalone

Version:

Version 1.1

Date: June 28, 2026

Contents

1	Layout and Formatting	1
1.1	Modular Document Structure	1
1.2	Custom Graphic Boxes	1
1.2.1	Definitions	1
1.2.2	Notes	2
1.2.3	Warnings	2
1.2.4	Numbered Examples	2
1.2.5	Italian Variants	2
1.3	Advanced Math Macros	3
1.3.1	Matrices and Systems	3
1.3.2	Differential Calculus	3
1.3.3	Signals and Transforms	3
1.3.4	Boolean Algebra	3
1.3.5	Complex Numbers and Phasors	4
1.3.6	Number Sets and Norms	4
1.4	Physical Units (SIunitx)	4
2	Source Code Listings	5
2.1	Hardware Code (VHDL)	5
2.1.1	Inline VHDL	5
2.2	Software and Scripting Languages	6
2.2.1	C and C++	6
2.2.2	Java	6

2.2.3	Python and MATLAB	6
3	Advanced Tools	7
3.1	Document Version (<code>\viewversion</code>)	7
3.2	Acronym Management	7
3.2.1	Configuration	7
3.2.2	Usage in Text	8
3.3	Bibliography (IEEE Standard)	8
3.4	Technical Diagrams with TikZ	8
3.4.1	Macro Syntax	8
3.4.2	Available Commands	9
3.4.3	Example: Hardware Datapath	9
3.4.4	Example: Software Flowchart	9
4	Quick Command Reference	10
4.1	Document Customisation	10
4.2	Graphic Boxes	11
4.3	Math Macros	11
4.4	Physical Units	12
4.5	Code Listings	12
4.6	TikZ Diagram Macros	13
	References	14

Chapter 1

Layout and Formatting

This chapter describes the formatting commands provided by the `stile_ingegneria.sty` package: custom graphic boxes, math macros for STEM disciplines, and physical unit formatting. All commands are available directly in your chapter `.tex` files with no additional declarations.

1.1 Modular Document Structure

The document is split into separate `.tex` files, one per chapter, stored in the `capitoli/` folder. The `main.tex` file acts as the orchestrator: it defines the preamble, declares acronyms, and includes chapters in the desired order via `\input`.

To add a new chapter, create the file in `capitoli/` and add the corresponding line to `main.tex`:

```
\input{capitoli/your_file_name.tex}
```

Warning: The order of `\input` lines in `main.tex` determines the chapter order in the final document. Section, figure, table, and example numbering is updated automatically by L^AT_EX on each recompilation.

1.2 Custom Graphic Boxes

The template provides four graphic environments to visually structure technical content. Each box is invoked with a compact one- or two-argument command, without interacting directly with `tcolorbox` or `mdframed`. All four commands have an Italian variant with a translated label, described in Section [1.2.5](#).

1.2.1 Definitions

To insert a formal definition with a blue background:

```
\definition{Box Title}{Definition text...}
```

Definition: Laplacian Filter

The Laplacian filter is a second-order differential operator used in image processing to detect edges (*edge detection*).

1.2.2 Notes

To insert a technical note or side remark with a lateral bar:

```
\note{Your note text...}
```

Note: *Ensure the sampling frequency satisfies the Nyquist-Shannon theorem ($f_s > 2 \cdot f_{max}$) to avoid aliasing artefacts.*

1.2.3 Warnings

To flag a common error, a critical constraint, or unexpected behaviour:

```
\warning{Your warning text...}
```

Warning: Do not prepend `\node` to TikZ macros: they already generate the graphic node internally. Adding `\node` before `\sysblock` (or any other macro) causes a compilation error.

1.2.4 Numbered Examples

To insert a practical example. The template numbers the box automatically based on the current chapter (e.g. Example 1.1):

```
\example{Optional Title}{Example text...}
```

Example 1.0 – Voltage Divider

With $V_{in} = 5\text{ V}$, $R_1 = 1\text{ k}\Omega$ and $R_2 = 2\text{ k}\Omega$, the output voltage is:

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2} = 3.33\text{ V}$$

1.2.5 Italian Variants

All four boxes have an Italian alias with a translated label. The commands below are equivalent — the language option set at package load (`[italian]` or `[english]`) automatically selects the correct label in the PDF.

English (primary)	Italian (alias)
<code>\definition{Title}{Text}</code>	<code>\definizione{Titolo}{Testo}</code>
<code>\note{Text}</code>	<code>\nota{Testo}</code>
<code>\warning{Text}</code>	<code>\attenzione{Testo}</code>
<code>\example{Title}{Text}</code>	<code>\esempio{Titolo}{Testo}</code>

Table 1.1: English and Italian box command equivalents.

1.3 Advanced Math Macros

The package includes a set of macros for the most common formula patterns in STEM. The goal is to reduce repetitive code and maintain consistent notation throughout the document.

1.3.1 Matrices and Systems

```
\[ K = \matrixthree{0}{-1}{0}{-1}{4}{-1}{0}{-1}{0} \]
\[ \system{ V_{out} } &= A \cdot (V^+ - V^-) \\ P &= V \cdot I } \]
```

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{cases} V_{out} = A \cdot (V^+ - V^-) \\ P = V \cdot I \end{cases}$$

1.3.2 Differential Calculus

```
\[ \der{f}{x}, \quad \pder{V}{t}, \quad \quad \eval{x^2}{0}{5} \]
```

$$\frac{df}{dx}, \quad \frac{\partial V}{\partial t}, \quad \left[x^2 \right]_0^5$$

1.3.3 Signals and Transforms

```
\[ \fourier{x(t)}, \quad \quad \laplace{h(t)}, \quad \quad \ztrans{x[n]} \]
```

$$\mathcal{F}\{x(t)\}, \quad \mathcal{L}\{h(t)\}, \quad \mathcal{Z}\{x[n]\}$$

1.3.4 Boolean Algebra

```
\[ Y = \notlog{A \cdot B} + C \]
```

$$Y = \overline{A \cdot B} + C$$

1.3.5 Complex Numbers and Phasors

`\[ \real{Z}, \quad \imag{Z}, \quad \fasore{220}{30^\circ} \]`

$$\Re\{Z\}, \quad \Im\{Z\}, \quad 220\angle 30^\circ$$

1.3.6 Number Sets and Norms

`\[ x \in \RR, \quad n \in \NN, \quad z \in \CC, \quad \norm{x} = 1 \]`

$$x \in \mathbb{R}, \quad n \in \mathbb{N}, \quad z \in \mathbb{C}, \quad \|x\| = 1$$

1.4 Physical Units (`SIunitx`)

The template loads `siunitx` and exposes its main command `\qty{value}{unit}`. Using this command — instead of typing units by hand — guarantees correct spacing, upright (non-italic) formatting, and consistency with the SI standard throughout the document.

The frequency is `\qty{150}{\MHz}` and the delay is `\qty{4.2}{\ns}`.

The frequency is 150 MHz and the delay is 4.2 ns.

Note: Compound units are built by chaining symbols with `\per`. For example, `\qty{9.81}{m\per s\squared}` produces 9.81 m s⁻². For the full list of predefined units refer to the official `siunitx` documentation.

Chapter 2

Source Code Listings

The template integrates the `listings` package with predefined styles for the most common languages in engineering. Each style handles keyword colouring, line numbering, spacing, and monospaced font automatically. An inline command is also available for each language to cite code fragments within body text.

2.1 Hardware Code (VHDL)

The `vhdl` style is based on an extended language definition (`VHDLext`) covering IEEE 1076-2008 keywords across four colour levels: structural statements, object declarations, IEEE types and libraries, functions and operators. To insert a VHDL block:

```
\begin{lstlisting}[style=vhdl, caption={Block description}]
-- Paste your VHDL code here
entity D_FlipFlop is
    port ( clk, d : in std_logic; q : out std_logic );
end entity;
\end{lstlisting}
```

```
1 entity D_FlipFlop is
2     port ( clk, d : in std_logic; q : out std_logic );
3 end entity;
```

Listing 2.1: VHDL style output.

Note: The `vhdl` style uses `columns=fullflexible` and `showstringspaces=false`: spacing faithfully follows the original source and no symbols are shown inside string spaces. This produces clean typographic output even with composite operators and signal names.

2.1.1 Inline VHDL

To cite a signal or instruction directly within body text, use the dedicated command:

Sampling occurs on `\vhdlinline{rising_edge(clk)}`.

Sampling occurs on `rising_edge(clk)`.

2.2 Software and Scripting Languages

For high-level languages the template uses a universal style (`UniversalStyle`) inspired by light IDE themes. The styles `c`, `cpp`, `java`, `python`, and `matlab` all inherit from it. The syntax is identical to VHDL — only the `style` parameter changes.

2.2.1 C and C++

```
1 #include <stdio.h>
2 int main() { return 0; }
```

Listing 2.2: Main function — C style.

Note: For C++ use `style=cpp`. For inline code use `\cinline{int x = 0;}` or `\cppinline{std::cout << x;}`.

2.2.2 Java

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("Hello");
4     }
5 }
```

Listing 2.3: Java class example.

The entry point is `public static void main`.

For inline code use `\javainline{...}`.

2.2.3 Python and MATLAB

```
1 import numpy as np
2 mean = np.mean(data)
```

Listing 2.4: Library import — Python style.

To compute the mean in Python use `np.mean(data)`.

Inline commands: `\pythoninline{...}` and `\matlabinline{...}`.

Chapter 3

Advanced Tools

This chapter documents the higher-level features of the template: document version management, automatic acronym handling, IEEE bibliography, and TikZ macros for technical diagrams.

3.1 Document Version (`\viewversion`)

The template includes a dedicated macro for version management, designed for technical documents that go through successive revisions. Override `\versiondoc` in `main.tex` inside the quick-customisation block:

```
\renewcommand{\versiondoc}{1.1}
```

Once set, the helper macro `\viewversion` inserts the string *Version X.X* or *Versione X.X* (depending on the package language option) anywhere in the document, including the cover page. The version is defined in one place and propagates everywhere it is called. The command `\mostraversione` is available as an alias for backwards compatibility.

Warning: Do not modify the original `\versiondoc` definition inside `stile_ingegneria.sty`. Always use `\renewcommand` in `main.tex`: this keeps the style file clean and independently updatable.

3.2 Acronym Management

The template uses the `acro` package for automatic acronym handling. The main advantage is that you never need to track manually where the first occurrence of an acronym appears — the system does it for you.

3.2.1 Configuration

Declare all acronyms in `main.tex`, before `\begin{document}`:

```
\DeclareAcronym{fpga}{short=FPGA, long=Field Programmable Gate Array}
```

3.2.2 Usage in Text

Call the acronym with `\ac{key}`. On the first occurrence the system automatically prints the full form followed by the abbreviation in parentheses — *Field Programmable Gate Array (FPGA)* — and uses only the abbreviation in all subsequent occurrences: FPGA.

Note: If after recompilation an acronym expands again when it should not, run a clean build (delete auxiliary files `.aux`, `.acn`, `.acr`) and recompile twice.

3.3 Bibliography (IEEE Standard)

The template uses `biblatex` with `biber` backend and `ieee` style: citations appear in the text as numbers in square brackets [1] and the bibliography section is generated and sorted automatically.

Workflow: add references to `layout/bibliografia.bib` in BibTeX format, then cite in the text with `\cite{key}`. To compile correctly run the sequence: `pdflatex` → `biber` → `pdflatex` → `pdflatex` (on Overleaf this happens automatically).

Note: The citation key (e.g. `turing1950`) can be any alphanumeric string without spaces. Choose a consistent convention such as `surname_year` or `surname_year_word`.

3.4 Technical Diagrams with TikZ

The template includes four TikZ macros that eliminate the need to write `\node[...]` code by hand. The macros produce graphic blocks consistent with the document's colour palette, connectable to each other via standard TikZ arrows.

3.4.1 Macro Syntax

All four macros share the same three-parameter signature:

```
\command{id}{label}{positioning}
```

- **id**: unique node identifier, used to connect it to other elements (e.g. `alu1`, `dec_valid`).
- **label**: text displayed inside the shape.
- **positioning**: relative TikZ coordinate (e.g. `right=of filter`, `below=1.5cm of start`). Leave empty (`{}`) for the first node of the diagram.

Warning: Do not prepend `\node` to the macros: they already generate the TikZ node internally. Adding `\node` before `\sysblock` (or any other macro) causes a compilation error.

3.4.2 Available Commands

Command	Shape	Typical use
<code>\startendblock</code>	Red rounded rectangle	Algorithm start / end
<code>\sysblock</code>	Blue rounded rectangle	Hardware blocks, registers
<code>\flowblock</code>	Orange rectangle	Software operations, processes
<code>\decblock</code>	Green diamond	Conditions, branches, loops

Table 3.1: TikZ macro reference.

3.4.3 Example: Hardware Datapath

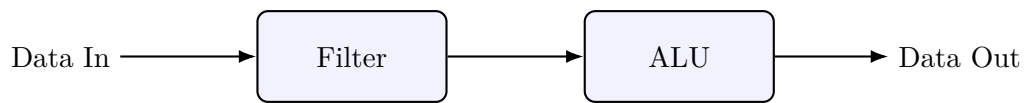


Figure 3.1: Three-stage hardware datapath example.

3.4.4 Example: Software Flowchart

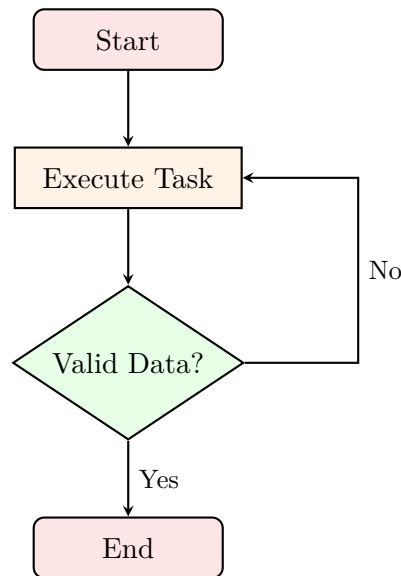


Figure 3.2: Software flowchart with retry loop.

Note: For the `tikzpicture` environment, always load the `arrow/.style={thick,->,>=stealth}` style and use `node distance` to control spacing between blocks. Relative positioning (`below=of`, `right=of`) is more robust than absolute coordinates when inserting or removing nodes.

Chapter 4

Quick Command Reference

This chapter collects all template commands in tabular form, organised by category. Use it as a quick reference while writing your document, without consulting previous chapters.

4.1 Document Customisation

Command	Where to use
<code>\renewcommand{\versiondoc}{X.Y}</code> Sets the document version shown on the cover page.	<code>main.tex</code>
<code>\viewversion</code> Prints <i>Version X.X</i> or <i>Versione X.X</i> at the call site, depending on the package language option.	Cover page / header
<code>\DeclareAcronym{key}{short=..., long=...}</code> Declares an acronym.	<code>main.tex</code> preamble
<code>\ac{key}</code> Inserts the acronym (expanded on first use).	Chapter text
<code>\input{capitoli/file.tex}</code> Includes a chapter file.	<code>main.tex</code> chapters section
<code>\cite{key}</code> Inserts a numeric IEEE citation.	Chapter text

Table 4.1: Document customisation commands.

4.2 Graphic Boxes

Command (EN)	Style	Typical use
<code>\definition{Title}{Text}</code>	Blue box with title	Formal definitions, theorems
<code>\note{Text}</code>	Grey-blue lateral bar	Technical notes, cross-references
<code>\warning{Text}</code>	Red lateral bar	Common errors, critical constraints
<code>\example{Title}{Text}</code>	Teal numbered box	Worked examples, calculations

Table 4.2: English box commands.

English	Italian equivalent
<code>\definition{Title}{Text}</code>	<code>\definizione{Titolo}{Testo}</code>
<code>\note{Text}</code>	<code>\nota{Testo}</code>
<code>\warning{Text}</code>	<code>\attenzione{Testo}</code>
<code>\example{Title}{Text}</code>	<code>\esempio{Titolo}{Testo}</code>

Table 4.3: EN/IT box command equivalents.

4.3 Math Macros

Command	Output	Category
<code>\matrixthree{a}{b}...{i}</code>	3×3 matrix	Linear algebra
<code>\system{eq1 \ \ eq2}</code>	Aligned system	Linear algebra
<code>\der{f}{x}</code>	$\frac{df}{dx}$	Differential calculus
<code>\pder{V}{t}</code>	$\frac{\partial V}{\partial t}$	Differential calculus
<code>\eval{f}{a}{b}</code>	$[f]_a^b$	Integral calculus
<code>\fourier{x(t)}</code>	$\mathcal{F}\{x(t)\}$	Transforms
<code>\laplace{h(t)}</code>	$\mathcal{L}\{h(t)\}$	Transforms
<code>\ztrans{x[n]}</code>	$\mathcal{Z}\{x[n]\}$	Transforms
<code>\notlog{A}</code>	$\overline{A}$	Boolean algebra
<code>\real{Z}</code>	$\Re\{Z\}$	Complex numbers
<code>\imag{Z}</code>	$\Im\{Z\}$	Complex numbers
<code>\fasore{V}{theta}</code>	$V\angle\theta$	Phasors
<code>\RR, \NN, \ZZ, \CC</code>	$\mathbb{R}, \mathbb{N}, \mathbb{Z}, \mathbb{C}$	Number sets
<code>\norm{x}</code>	$\ x\ $	Vector spaces

Table 4.4: Math macro reference.

4.4 Physical Units

Command	Output	Notes
<code>\qty{150}{\MHz}</code>	150 MHz	Simple unit
<code>\qty{4.2}{\ns}</code>	4.2 ns	Prefixed unit
<code>\qty{9.81}{\m\per\s\squared}</code>	9.81 m s ⁻²	Compound unit
<code>\qty{3.3}{\V}</code>	3.3 V	Volt
<code>\qty{1}{\kohm}</code>	1 kΩ	Ohm with prefix

Table 4.5: SIunitx command examples.

4.5 Code Listings

style=	Language	Notes
vhdl	VHDL	IEEE 1076-2008, 4 keyword levels
c	C	Universal light style
cpp	C++	Universal light style
java	Java	Universal light style
python	Python	Universal light style
matlab	MATLAB	Universal light style

Table 4.6: Available listing styles.

Basic syntax (same for all languages):

```
\begin{lstlisting}[style=python, caption={Listing description}]
% your code here
\end{lstlisting}
```

Inline command	Language
<code>\vhdlinline{...}</code>	VHDL
<code>\cinline{...}</code>	C
<code>\cppinline{...}</code>	C++
<code>\javainline{...}</code>	Java
<code>\pythoninline{...}</code>	Python
<code>\matlabinline{...}</code>	MATLAB

Table 4.7: Inline code commands.

4.6 TikZ Diagram Macros

Macro	Shape	Typical use
<code>\startendblock{id}{label}{pos}</code>	Red rounded rect.	Start / end of algorithm
<code>\sysblock{id}{label}{pos}</code>	Blue rounded rect.	Hardware blocks, registers
<code>\flowblock{id}{label}{pos}</code>	Orange rectangle	Software ops, processes
<code>\decblock{id}{label}{pos}</code>	Green diamond	Conditions, branches

Table 4.8: TikZ macro reference.

Recommended `tikzpicture` environment for flowcharts:

```

\begin{figure}[H]
\centering
\begin{tikzpicture}[node distance=1cm,
  arrow/.style={thick,->,>=stealth}]
  % nodes
  \startendblock{start}{Start}{};
  \flowblock{step1}{Operation}{below=of start};
  % arrows
  \draw[arrow] (start) -- (step1);
\end{tikzpicture}
\caption{Diagram description.}
\end{figure}

```


References

- [1] A. Turing, “Computing machinery and intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950.